

BEACON DATA PROCESSING SYSTEM (BPS)**SPECIFICATION****TEAM MEMBERS:**

JIM PHAM

BRUCE DAVIS,

EDWARD ANTHONYSAMY,

VEERARAGAVAN GOPALAKRISHNAN,

DAVID TOPHAM

Document Information

Author	Version Date	Change Summary
Jim Pham	May 2, 2008	Initial draft - V 0.1
Edward Anthonysamy	May 11, 2008	Draft - V 0.2
Veeraragavan Gopalakrishnan	May 12, 2008	Draft – V 0.3
Bruce	May 22, 2008	Draft – V 0.4
Edward Anthonysamy	May 23, 2008	Draft – V 0.5

Contents

DOCUMENT INFORMATION	1
CONTENTS	2
INTRODUCTION	3
BPS SYSTEMS	3
Beacon Client Application	3
Beacon Web Services	3
Beacon Data Format	3
Beacon Data Service	4
FEATURE REQUIREMENTS	4
TECHNOLOGY	4
DESIGN	5
Architecture Model	5
Database Schema	6
BEACON CLIENT APPLICATION	6
BEACON WEB SERVICES	21
BEACON DATA FORMAT	22
BEACON DATA SERVICES	23
DEPLOYMENT	24
Web Server Configuration	25
MySQL Configuration	26
Reporting Query	26

Introduction

The Beacon Processing System (BPS) is a web-based application that provides the ability for external beacon operators to submit data, as well as external operators, educators, and public access to the archived beacon data. This project was developed in partnership with Santa Clara University for CREST/NASA.

BPS Systems

BPS consists of the following components:

- Beacon Client Application (CREST)
- Beacon Web Services
- Beacon Data Format
- Beacon Data Service
- MySQL

Beacon Client Application

Beacon Client Application is a Web-based application using the .NET ASPX model.

ASPX is essentially a web page that implements the .NET Web Form and C#.

Beacon Web Services

- Beacon Web Services (WS) are a set collection of public APIs that are responsible for the beacon data format, decoding, and performing the business logic.
- Beacon Web Services use an industry-specific messaging standard (SOAP) to be interoperable between the beacon client application and beacon data format.
- These public WS can also be used by (developer, NASA, school, etc) to perform the business logic.

Beacon Data Format

- This module is the business logic for the beacon data format, such as decoding, converting, calculating, etc.
- Before sending the beacon data to the database, the beacon data is first decoded, converted, and calculated.
- This module works along with the beacon Web services Module and beacon data service modules.

Beacon Data Service

- Beacon Data Service is responsible for the data access that includes database connectivity as well as accessing the database.
- This module is the data tier of the beacon data processing system.

Feature Requirements

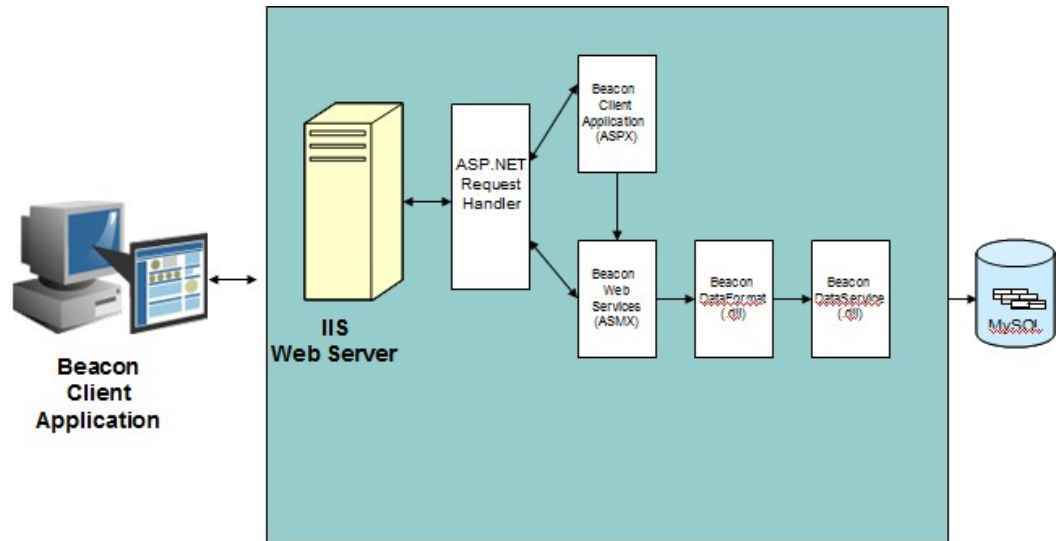
1. The BPS accepts submitted beacon reports (that will include message data, callsign, and other data) and returns a QSL page to the user.
2. The BPS submits message data and submits the messages in a MySQL database.
3. The BPS provides summary reports on the contents of the message archive, to include the total number of messages, the number of unique HAM operator callsigns, the number of unique countries from which operators submit, and other data.
4. BPS provides basic message archive search capability.
5. The BPS provides summary reports on requested access to the message archive, to include the total number of searches, characteristics of those requesting data, and other data.

Technology

BPS uses Visual Studio 2008 and the .NET Framework by Microsoft.

Design

Architecture Model



Database Schema

RawData		
U1	RawSeqID ContributorSeqID RawData RawTimeStamp	INTEGER VARCHAR(6) LONGVARCHAR DATETIME

PacketFormat		
	PFSatelliteID PFFieldName	VARCHAR(20) VARCHAR(50)
U1	PFSeqID PFStartingPosition PFFieldLength PFWellNumber PFFieldType PFFieldUnit PFCalibrationOffset	INTEGER INTEGER INTEGER VARCHAR(4) VARCHAR(20) VARCHAR(20) INTEGER

Contributors		
PK	<u>ContributorSeqID</u>	INTEGER
I1	FullName	VARCHAR(100)
	CallSign	VARCHAR(6)
I2	EmailAddress	VARCHAR(100)
	Location	VARCHAR(100)
	Password	VARCHAR(10)

PacketHeader		
U1	PacketSeqID RawSeqID PHWebSite PHBusTime PHHealthPowerStatus PHHealthStartupCounter PHHealthSpacecraftID PHHealthExperimentPhase PHWellNumber	INTEGER INTEGER CHAR(14) INTEGER INTEGER INTEGER INTEGER INTEGER INTEGER

PacketReadings		
U2	PacketSeqID	INTEGER
U2	PRTType	VARCHAR(20)
	PRData	INTEGER

SubmissionComments		
	PacketSeqID	INTEGER
I1	ContributorSeqID Comments	INTEGER LONGVARCHAR

Codes		
PK	<u>CodeType</u>	VARCHAR(10)
PK	<u>CodeID</u>	VARCHAR(10)
	CodeDescription	VARCHAR(50)

Beacon Client Application

Beacon Client Application has been developed using Visual Studio 2005 and ASP.NET C# has been used as a programming language to develop the web forms.

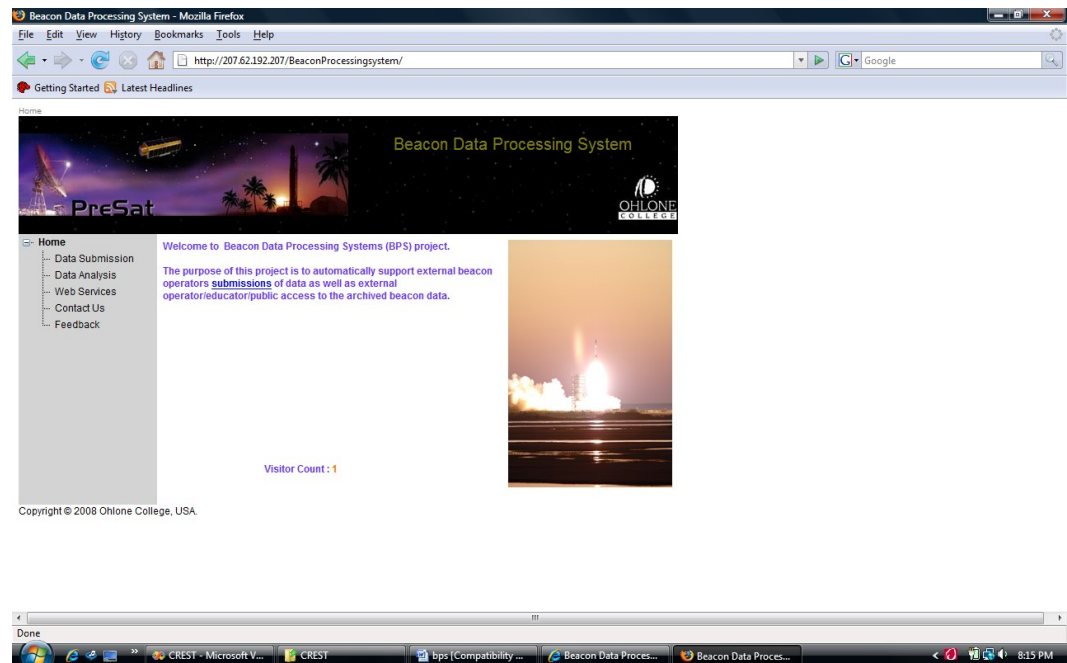
The client application has one Master page layout, which is used across the child forms. The child forms are built using a Content page layout which merges with the Master page layout.

The Beacon Client Application includes the following features on the main default page.

- Data Submission Form
- Data Analysis Form
- List of Web Services Form

- Contact Us Form
- Feedback Form

The general layout of the default main page is as follows



The Master page contains header images, such as PreSat & Ohlone College images, that are stored in the ~/Images folder of the client solution, i.e., BeaconClientApp. All images associated with the project should be stored in the ~/Images folder.

The Master page contains a TreeView control on the left side of the pane, which gets the value for navigation from the Web.sitemap XML file.

The contents of the Web.sitemap xml file are as follows

```
<?xml version="1.0" encoding="utf-8" ?>
<siteMap xmlns="http://schemas.microsoft.com/AspNet/SiteMap-File-1.0" >
  <siteMapNode url="Default.aspx" title="Home" description="">
```

```

<siteMapNode url="Submission.aspx" title="Data Submission"
description="Beacon data entry Page" />
<siteMapNode url="Search.aspx" title="Data Analysis"
description="Beacon data analysis Page" />
<siteMapNode url="Services.aspx" title="Web Services"
description="Web Services List Page" />
<siteMapNode url="Contactus.aspx" title="Contact Us"
description="Contact Us Page" />
<siteMapNode url="Feedback.aspx" title="Feedback"
description="Feedback Page" />
</siteMapNode>
</siteMap>

```

Data Submission Form

The screenshot shows a web browser window titled "Beacon Data Processing System - Mozilla Firefox". The address bar shows the URL "http://207.62.192.207/BeaconProcessingSystem/Submission.aspx". The page has a navigation menu on the left with links: Home, Data Submission (selected), Data Analysis, Web Services, Contact Us, and Feedback. The main content area has a header banner for "Beacon Data Processing System" with "PreSat" and "OHIO ONE COLLEGE" logos. Below the banner, a message says: "Thank you for your interest in our mission. If you have received beacon messages from the PreSat / PharmaSat spacecraft, please use the form below to submit the messages. You will receive the confirmation about your submission of success in a QSL page or any errors below on the status column." The form includes fields for Name, Email, Call Sign, and Location (a dropdown menu with "Select one..."). Below these is a section titled "Beacon Packets Entry" with a "HyperLink" for more information and a large yellow text area for packet data. At the bottom, there is a text area for additional comments, "Submit" and "Reset" buttons, and a "Status of Submission:" label. The footer shows "Copyright © 2008 Ohlone College, USA." and a taskbar with various application icons.

The controls present in the Submission form are as follows

1. Name

Description: Name of the person who is submitting the beacon packets

Control: Textbox control

Validation: Required

2. Email

Description: Email ID of the person who is submitting the beacon packets.

Control: Textbox control

Validation: Required and has to conform to an email regular expression.

3. Call Sign

Description: Call Sign associated with the submission.

Control: Textbox control

Validation: Required and has to be a 6-digit alphanumeric character

4. Location

Description: The location from where the user is submitting the beacon packets.

Control: Dropdown list

Validation: Required and has to be one of the country names from the dropdown list.

5. Beacon Packets Entry

Description: The placeholder to enter the beacon packets.

Control: Multiline Textbox

Validation: Required

6. Additional Comments Entry

Description: The placeholder to enter any additional comments, such as signal strength.

Control: Textbox

Validation: Optional

7. Status of Submission

Description: The placeholder to indicate any errors associated with the packet submission.

Control: Label

Validation: N/A

8. Submit

Description: The control submits data back to the server for processing. If there are no errors, the user will be redirected to a successful QSL page.

Control: Button

Validation: N/A

9. Reset

Description: The control used to reset the input values on the controls.

Control: Button

Validation: N/A

There is a hyperlink present in the form which the user can click to get access to information about GeneSat-1 Beacon packet decoding.

Data Analysis Form

The controls present in the Data Analysis form are as follows

1. Name

Description: Name of the person who is requesting the beacon data fields for analysis

Control: Textbox control

Validation: Required

2. Email

Description: Email ID of the person who is requesting the beacon data fields.

Control: Textbox control

Validation: Required and has to conform to a regular email expression.

3. Requester Type

Description: Type of user who is requesting the data.

Control: Dropdown list

Validation: Required and has to be one of the four valid values from the dropdown list

4. Requester Location

Description: The location from where the user is requesting the beacon packet fields.

Control: Dropdown list

Validation: Required and has to be one of the country names from the dropdown list.

5. Satellite

Description: The satellite from which the data is requested.

Control: Dropdown list

Validation: Required and has to be one of the listed Satellites from the dropdown list.

6. Requested Fields

Description: The beacon packet fields which can be queried for data analysis.

Control: Dropdown list

Validation: Required and can be a multiselect list

7. From Date

Description: The starting date from which the beacon data fields need to be retrieved from the database.

Control: Textbox

Validation: Required and has to be of the data format MM/DD/YYYY

8. To Date

Description: The end date to which the beacon data fields need to be retrieved from the database.

Control: Textbox

Validation: Required and has to be of the data format MM/DD/YYYY

10. Submit

Description: The control submits data back to the server for processing.

Control: Button

Validation: N/A

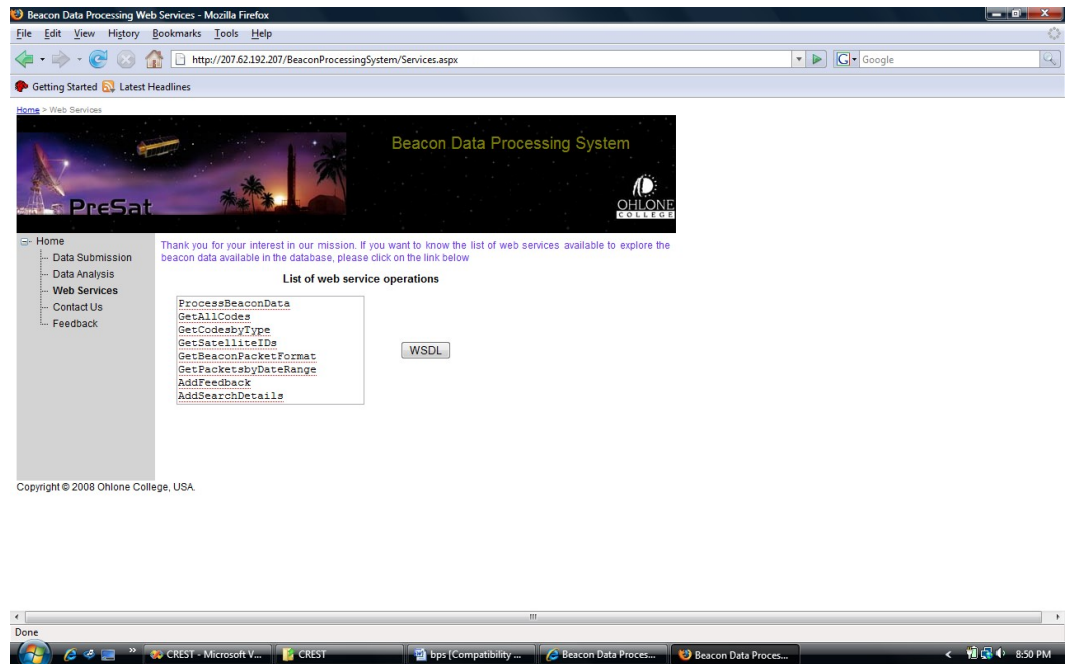
11. Reset

Description: The control used to reset the input values on the controls.

Control: Button

Validation: N/A

Web Services Form



The controls present in the Web Services form are as follows

1. Web services List

Description: Control used to display the list of methods available as web services.

Control: Multiline text box.

Validation: N/A. It is a read-only control.

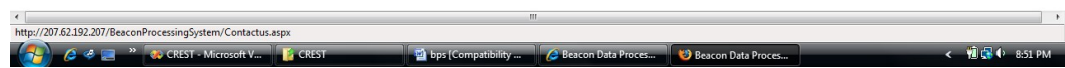
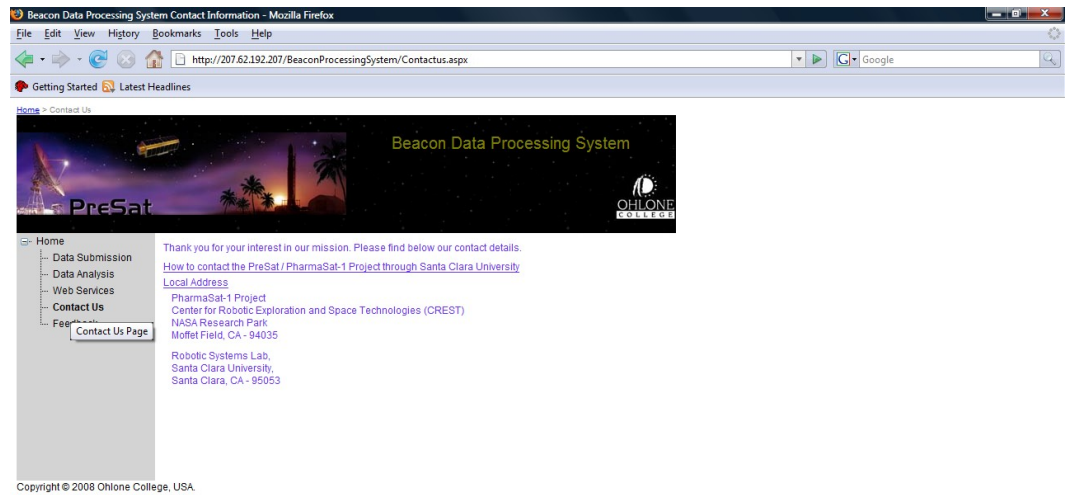
2. WSDL

Description: The control used to invoke the WSDL (Web Services Description Language) page associated with the web services.

Control: Button

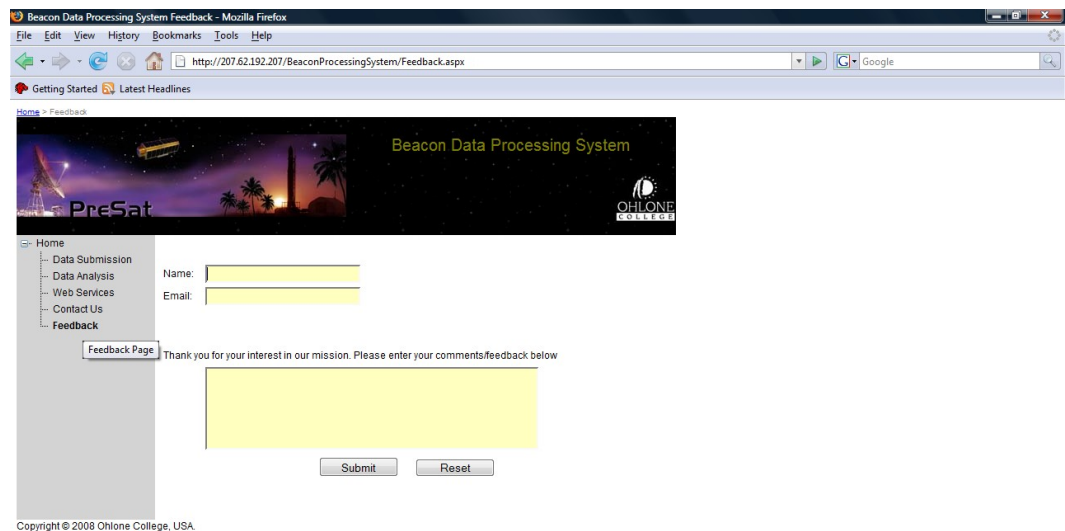
Validation: N/A

Contact Us Form



The Contact Us form contains a placeholder to enter contact details as free text in the cells present in the table layout of the web form.

Feedback Form



The controls present in the Feedback form are as follows

1 Name

Description: Name of the person who is submitting the feedback

Control: Textbox control

Validation: Required

2 Email

Description: Email Id of the person who is submitting the feedback.

Control: Textbox control

Validation: Required and has to conform to a regular email expression.

3 Feedback Entry

Description: Placeholder to enter the feedback.

Control: Multiline text box.

Validation: Required

4 Submit

Description: The control used to submit feedback data back to the server for processing. Upon successful saving of the feedback, the user will be notified about the status at the bottom of the form in the Status Result label control.

Control: Button

Validation: N/A

5 Reset

Description: The control used to reset the input values on the controls.

Control: Button

Validation: N/A

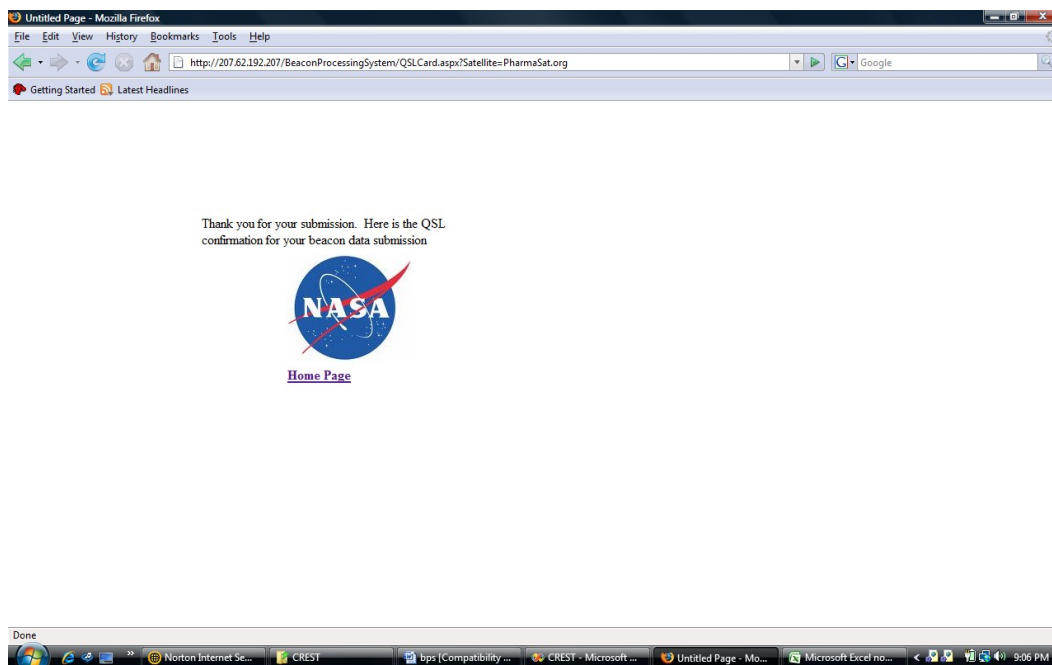
6 Status Result

Description: The control used to display the status of the save or any errors.

Control: Label

Validation: N/A and visible only upon successful saving or error display, and it is not visible during the normal page load.

QSL Form

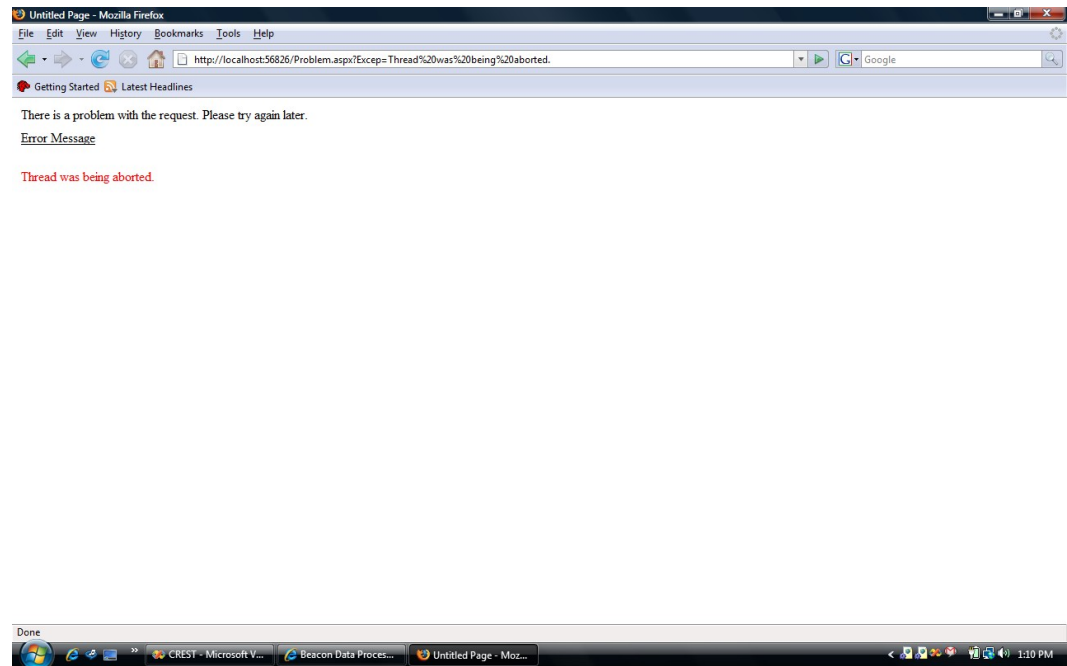


The QSL form will be displayed when the user successfully inserts a beacon packet into the database.

The image of the QSL confirmation can be customized using the configuration settings in the web.config file in BeaconClientApp.

Currently, a NASA.jpg image is in the ~/Images folder and is used as the QSL image for both PreSat and PharmaSat beacon data submissions.

Problem Form



The Problem form will be displayed when a generic error occurs in the application.

Web.config

The contents of the Web.config file present in the BeaconClientApp is as follows

```
<?xml version="1.0"?>
<configuration>

  <configSections>
    <sectionGroup name="applicationSettings"
type="System.Configuration.ApplicationSettingsGroup, System,
Version=2.0.0.0, Culture=neutral,
PublicKeyToken=b77a5c561934e089" >
      <section name="CREST.Properties.Settings"
type="System.Configuration.ClientSettingsSection, System,
Version=2.0.0.0, Culture=neutral,
PublicKeyToken=b77a5c561934e089" requirePermission="false" />
    </sectionGroup>
  </configSections>
  <appSettings>
<!--
```

The QSL image jpg files can be manipulated depending on the satellite based on the key values below. Modify the image

file name present under Images directory of the client application. For now everything points to Nasa.jpg file name.

```
-->

    <add key="PharmaSat.orgQSLImage" value="~\Images\Nasa.jpg"/>
    <add key="PreSat.orgQSLImage" value="~\Images\Nasa.jpg"/>
    <add key="NanoSailD.orgQSLImage" value="~\Images\Nasa.jpg"/>
</appSettings>
<connectionStrings/>

<system.web>
  <!--
      Set compilation debug="true" to insert debugging
      symbols into the compiled page. Because this
      affects performance, set this value to true only
      during development.
  -->
  <compilation debug="true">
    <assemblies>
      <add assembly="System.Design, Version=2.0.0.0,
Culture=neutral, PublicKeyToken=B03F5F7F11D50A3A"/>
      <add assembly="System.Windows.Forms, Version=2.0.0.0,
Culture=neutral, PublicKeyToken=B77A5C561934E089"/>
    </assemblies>
  </compilation>
  <!--
      The <authentication> section enables configuration
      of the security authentication mode used by
      ASP.NET to identify an incoming user.
  -->
  <authentication mode="Windows"/>
  <!--
      The <customErrors> section enables configuration
      of what to do if/when an unhandled error occurs
      during the execution of a request. Specifically,
      it enables developers to configure html error pages
      to be displayed in place of a error stack trace.

      <customErrors mode="RemoteOnly"
defaultRedirect="GenericErrorPage.htm">
        <error statusCode="403" redirect="NoAccess.htm" />
        <error statusCode="404" redirect="FileNotFound.htm"
/>
      </customErrors>
  -->
</system.web>
<applicationSettings>
  <CREST.Properties.Settings>
    <setting name="CREST_localhost_BeaconWebService"
serializeAs="String">
<value>http://207.62.192.207/beaconwebservices/beaconwebservices.
asmx</value>
    </setting>
  </CREST.Properties.Settings>
</applicationSettings>
</configuration>
```

Beacon Web Services

Beacon Webservices provides the methods to store and retrieve data from the backend database. Beacon Client applications always invoke only the web services' methods to perform any kind of database-related operation. We have developed webservices in visual studio 2005 and using C# as the programming language. All the errors encountered in this layer are returned as soapexception to the caller.

Following methods are available in the Beaconwebservices:

The AddFeedback method is used to store the fields on the feedback page when the user submits feedback.

The AddSearchDetails method is used to store the user's search criteria for retrieving beacon packet information.

GetAllCodes method is used to retrieve all the records from the Codes table.

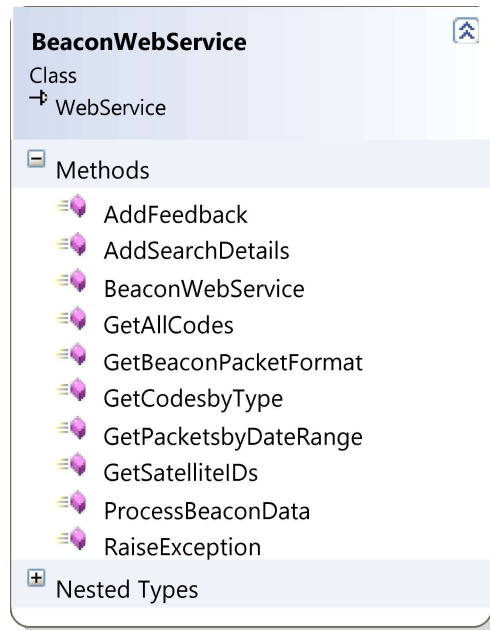
GetBeaconPacketFormat retrieves the beacon packet format information from the PackerFormat table for a specific satelliteID. SatelliteID is the input for this method.

The GetCodesbyType method is used to retrieve all records for a particular code type. For example, you can pass "Country" to this method to retrieve all the country codes.

GetPacketsbyDateRange method is used to retrieve the packet information for a specific date range. Complete details for the packet within the specified date range are returned to the caller, and it is up to the caller to filter out unwanted fields.

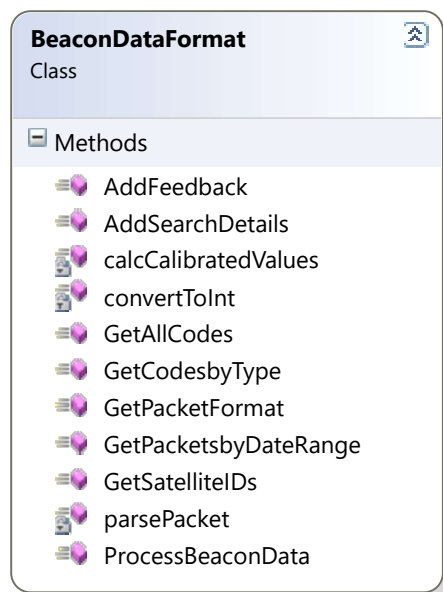
GetSatelliteIDs method is used to retrieve unique satelliteids from the packetformat table.

ProcessBeaconData method is used to store the contributor and packet informations submitted by the user.



Beacon Data Format

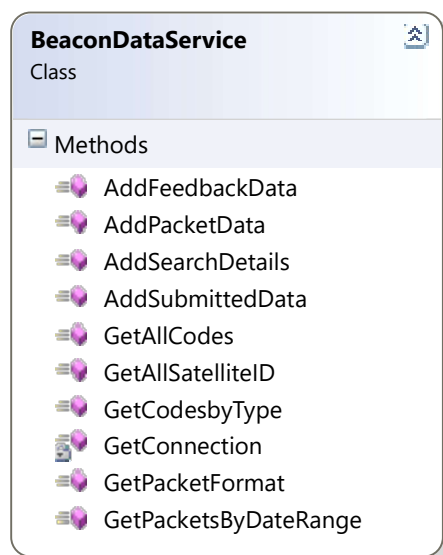
This is a business layer which contains methods to parse, converts the hexadecimal value coded in circular endian to integer value and calculates the engineering values of the fields in the beacon packet. It extracts the beacon packet from the input message submitted by the user and parses it based on the format retrieved from the packetformat table. Convert the hexadecimal values of the packer fields to integer values based on the circular endian formula and store it in the database by invoking the respective function in the dataservice layer. Integer values of the packet fields are converted to engineering values based on the calibrated slope and offset in the packetformat table when the user retrieves the packets for analysis. This acts as a liaison between the web service layer and the data service layer.



Beacon Data Services

This is the data service layer which provides all the functionalities to store, retrieve and update data in the database server. If any change arises to use different databases instead of MYSQL in the future, then this is the only layer that needs to be changed.

Methods in this layer are named to match those in other layers, making it easy to identify dependencies across layers. Exceptions in this layer are thrown back to the caller.



Deployment

Implementation Procedure:

I. Un-installation of previous version

If previous Beacon Processing System web application and/or web services exist, then perform the following section; otherwise, proceed to the next section.

1. **Unconfigure Beacon Processing System with Application Pool:**
 - a. Open the IIS Administrator Console
 - b. Locate BeaconProcessingSystem, right-click, select Properties, update the AppPool to use the DefaultAppPool, and click OK.
2. **Unconfigure Beacon Web Services with Application Pool:**
 - a. Open the IIS Administrator Console
 - b. Locate BeaconWebServices, right-click, select Properties, update the AppPool to use the DefaultAppPool, and click OK.
3. **Remove applications from the server:**
 - a. Go to Control Panel and select Add and Remove Programs.
 - b. Locate BeaconWebServices application, then click Remove.
 - c. Locate BeaconProcessingSystem application, then click Remove.

II. Installation

1. Un-zip the InstallBeacon_V1.zip file to the local server c:\temp directory.
2. **Install BeaconProcessingSystem Web application:**
 - a. Go to c:\temp\BuildXXXXXX\BeaconWebApp and run CRESTClientSetup.msi to install BeaconProcessingSystem web application:
 - b. Accept the defaults from the installation wizard.
 - c. Following all the steps to the end to complete installation.
3. **Install BeaconWebServices:**
 - a. Go to c:\temp\ BuildXXXXXX\BeaconWebservices and run BeaconWebServicesSetup.msi to install BeaconWebServices application.
 - b. Accept the defaults from the installation wizard.
 - c. Following all the steps to the end to complete installation.

III. Back out Procedure

If the decision is made to back out PEMM Member Enrollment Service, then perform the actions from section I.

Web Server Configuration

I. Beacon Web Application and WebServices Configuration

1. Open IIS Manager and expand Web Sites → Default Web Site.
2. In the AppPool node, verify existence of, or create the CrestWebPool
Accept the default settings for the application pool.
3. In the AppPool node, verify existence of, or create the CrestAppPool
Accept the default settings for the application pool.
4. Locate BeaconProcessingSystem under Default Web Site, then right click to open property:
 - a. On the Directory Tab, select the CrestWebPool as the application Pool.
 - b. On the Directory Security Tab, check Enable Anonymous Access.
 - c. On the ASP.NET tab, select version 2.0.
5. Locate BeaconWebServices under Default Web Site, then click to open property:
 - a. On the Directory Tab, select the CrestAppPool as the application Pool.
 - b. On the Directory Security Tab, check Enable Anonymous Access.
 - c. On the ASP.NET tab, select version 2.0.
6. Update BeaconWebServices url at the end of the Web.config file of BeaconProcessingSystem. Web.config file is available at the following location: **C:\Inetpub\wwwroot\BeaconProcessingSystem**

Values that need to be changed are highlighted in red.

```
<applicationSettings>
```

```
<CREST.Properties.Settings>
```

```
<setting name="CREST_localhost_BeaconWebService"
serializeAs="String">
```

```
<value>http://XXX.XX.X.XXX/beaconwebservices/beaconwebservices.a
smx</value>
```

```
</setting>
```

```
</CREST.Properties.Settings>
```

```
</applicationSettings>
```

7. Update the MYSQL database server IP address, username and password in the Web.config file of the BeaconWebServices. Web.config file is available at the following locations:

C:\Inetpub\wwwroot\BeaconWebServices

Values that need to be changed are highlighted in red.

```
<appSettings>
    <add key="dbConnStr"
value="server=localhost;uid=username;password=password;database=n
asa_crest"/>
</appSettings>
```

MySQL Configuration

MySQL version 5.1.24 was used, and the MySQL connector was version 5.2.1.

Reporting Query

To select beacon data submission count by Satellite

```
SELECT

packetheader.PHWebSite as Satellite,

COUNT(*) AS Counts

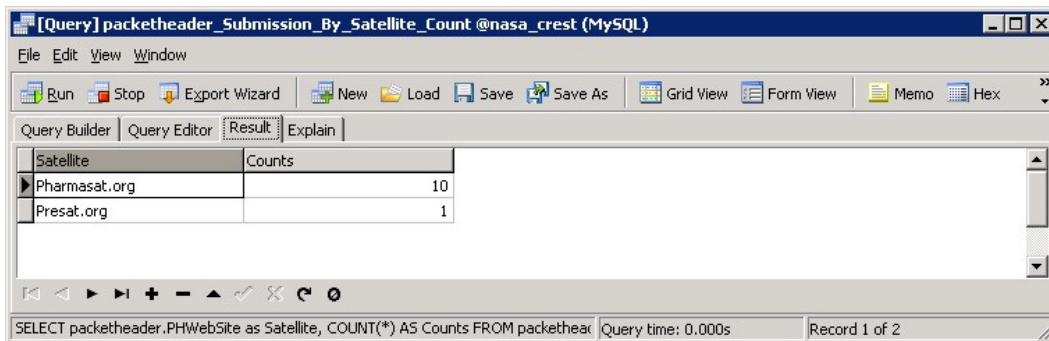
FROM

packetheader

GROUP BY

packetheader.PHWebSite

;
```



The screenshot shows a MySQL query execution window titled "[Query] packetheader_Submission_By_Satellite_Count @nasa_crest (MySQL)". The window has a menu bar (File, Edit, View, Window) and a toolbar with icons for Run, Stop, Export Wizard, New, Load, Save, Save As, Grid View, Form View, Memo, and Hex. Below the toolbar are tabs for Query Builder, Query Editor, Result, and Explain. The Result tab is active, displaying a table with two columns: Satellite and Counts. The table contains two rows: Pharmasat.org with a count of 10, and Presat.org with a count of 1. At the bottom of the window, the executed query is shown: "SELECT packetheader.PHWebSite as Satellite, COUNT(*) AS Counts FROM packetheader", along with the query time (0.000s) and the current record (Record 1 of 2).

Satellite	Counts
Pharmasat.org	10
Presat.org	1

To get the counts of searches done so far on any dates

```

SELECT

date(searchstats.SSCreateDateTime) as Search_Date,

COUNT(*) AS Counts

FROM

searchstats

GROUP BY

date(searchstats.SSCreateDateTime)

;

```

Search_Date	Counts
2008-05-11	10
2008-05-12	2
2008-05-23	8

To get the minimum & maximum date for FROM & TO date search pattern

```

SELECT

searchstats.SSName as Requester_Name,

searchstats.SSEmail as Requester_Email,

searchstats.SSRequesterType as Requester_Type,

searchstats.SSRequesterLocation as Requester_Location,

searchstats.SSatelliteID as SatelliteID,

date(searchstats.SSFromDate) AS Min_From_Date,

" as Max_From_Date,

" as Min_To_Date,

```

" as Max_To_Date

FROM

searchstats

where date(searchstats.SSFromDate) = (SELECT min(date(searchstats.SSFromDate)) from
searchstats)

union

SELECT

searchstats.SSName as Requester_Name,

searchstats.SSEmail as Requester_Email,

searchstats.SSRequesterType as Requester_Type,

searchstats.SSRequesterLocation as Requester_Location,

searchstats.SSatelliteID as SatelliteID,

" as Min_From_Date,

date(searchstats.SSFromDate) AS Max_From_Date,

" as Min_To_Date,

" as Max_To_Date

FROM

searchstats

where date(searchstats.SSFromDate) = (SELECT max(date(searchstats.SSFromDate)) from
searchstats)

union

SELECT

searchstats.SSName as Requester_Name,

searchstats.SSEmail as Requester_Email,

searchstats.SSRequesterType as Requester_Type,

searchstats.SSRequesterLocation as Requester_Location,

searchstats.SSatelliteID as SatelliteID,

" as Min_From_Date,

```

" as Max_From_Date,

date(searchstats.SSToDate) AS Min_To_Date,

" as Max_To_Date

FROM

searchstats

where date(searchstats.SSToDate) = (SELECT min(date(searchstats.SSToDate)) from searchstats
)

union

SELECT

searchstats.SSName as Requester_Name,

searchstats.SSEmail as Requester_Email,

searchstats.SSRequesterType as Requester_Type,

searchstats.SSRequesterLocation as Requester_Location,

searchstats.SSatelliteID as SatelliteID,

" as Min_From_Date,

" as Max_From_Date,

" as Min_To_Date,

date(searchstats.SSToDate) AS Max_To_Date

FROM

searchstats

where date(searchstats.SSToDate) = (SELECT max(date(searchstats.SSToDate)) from
searchstats )

;

```

[Query] search_Minimum_Maximum_From_To_Date @nasa_crest (MySQL)

File Edit View Window

Run Stop Export Wizard New Load Save Save As Grid View Form View Memo Hex Image

Query Builder Query Editor Result Explain

Requester_Name	Requester_Email	Requester_Type	Requester_Location	SatelliteID	Min_From_Date	Max_From_Date	Min_To_Date	Max_To_Date
tester2	tester3@gmail.com	Student	EC	PharmaSat.org	2000-01-01			
test	test@gmail.com	Student	SH	PharmaSat.org		2008-05-11		
dsdcd	asas@saas.com	Student	SH	PreSat.org			2008-02-20	
tester3	tester3@gmail.com	Student	EC	PharmaSat.org				2008-05-20
tester12	tester12@yahoo.com	Student	KN	PreSat.org				2008-05-20

SELECT searchstats.SSName as Requester_Name, searchstats.SSEmail as Requester_Email, searchstats.SSRequesterType as Requester_Type, searchstats.SSRequesterLocati Query time: 0.000s Record 1 of 7

To get the search requester Email counts

```

SELECT

searchstats.SSEmail AS Requester_Email,

COUNT(*) AS Counts

FROM

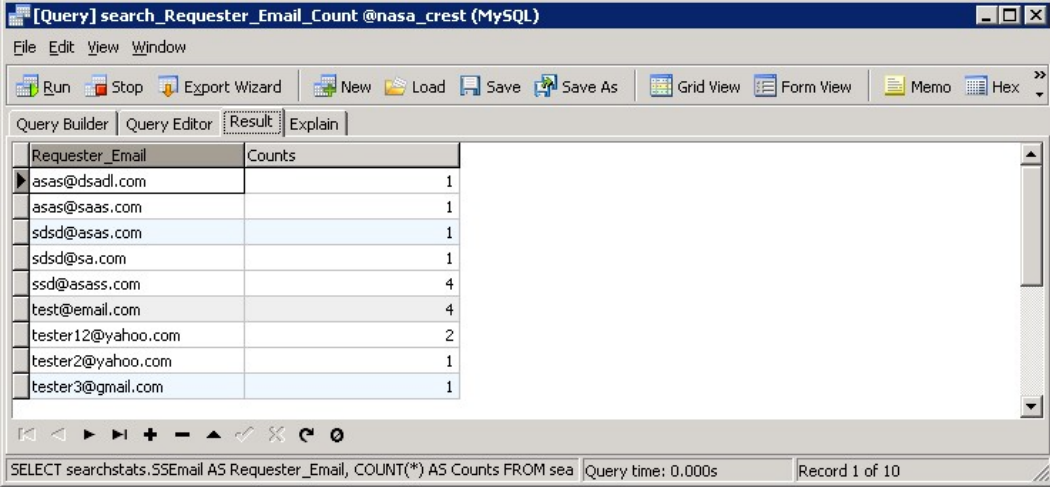
searchstats

GROUP BY

searchstats.SSEmail

;

```



The screenshot shows a MySQL query result window titled "[Query] search_Requester_Email_Count @nasa_crest (MySQL)". The window displays a table with two columns: "Requester_Email" and "Counts". The table contains 10 rows of data. The status bar at the bottom indicates the query was executed successfully, with a query time of 0.000s and 10 records returned.

Requester_Email	Counts
asas@dsadl.com	1
asas@saas.com	1
sdsd@asas.com	1
sdsd@sa.com	1
ssd@asass.com	4
test@email.com	4
tester12@yahoo.com	2
tester2@yahoo.com	1
tester3@gmail.com	1

To get the search requester Location counts

```

SELECT

codes.CodeDescription as Location_Name,

COUNT(*) AS Counts

FROM

searchstats ,

codes

WHERE

searchstats.SSRequesterLocation = codes.CodeID AND

```

```
codes.CodeType = 'Country'
```

```
GROUP BY
```

```
searchstats.SSRequesterLocation
```

```
;
```

Location_Name	Counts
Belgium	1
Ecuador	1
Falkland Islands (Malvinas)	1
Saint Kitts and Nevis	2
Mexico	4
Saint Helena	11

SELECT codes.CodeDescription as Location_Name, COI Query time: 0.000s Record 1 of 6

To get the search requester Name & Email counts

```
SELECT
```

```
searchstats.SSName AS Requester_Name,
```

```
searchstats.SSEmail AS Requester_Email,
```

```
COUNT(*) AS Counts
```

```
FROM
```

```
searchstats
```

```
GROUP BY
```

```
searchstats.SSName,
```

```
searchstats.SSEmail
```

```
;
```

Requester_Name	Requester_Email	Counts
dsdsd	asas@saas.com	1
dsdsd	sdsd@asas.com	1
dwdw	asas@dsadl.com	1
sdsd	ssd@asass.com	4
search test	test@email.com	1
ssd	sdsd@sa.com	1
test	test@email.com	3
tester12	tester12@yahoo.com	2
tester2	tester2@yahoo.com	1
tester3	tester3@gmail.com	1
tester5	tester5@netscape.com	4

SELECT searchstats.SSName AS Requester_Name, se Query time: 0.000s Record 1 of 11

To get the search requester Type counts

SELECT

searchstats.SSRequesterType AS Requester_Type,

COUNT(*) AS Counts

FROM

searchstats

GROUP BY

searchstats.SSRequesterType

;

Requester_Type	Counts
Public	4
Student	16

SELECT searchstats.SSRequesterType AS Requester_ Query time: 0.000s Record 1 of 2

To get the search requester Type , Location & Date of Search counts

```

SELECT

searchstats.SSRequesterType AS Requester_Type,

codes.CodeDescription AS Location_Name,

searchstats.SSatelliteID as SatelliteID,

date(searchstats.SSCreateDateTime) as Date_of_search,

COUNT(*) AS Counts

FROM

searchstats ,

codes

WHERE

searchstats.SSRequesterLocation = codes.CodeID AND

codes.CodeType = 'Country'

GROUP BY

searchstats.SSRequesterType,

searchstats.SSatelliteID,

date(searchstats.SSCreateDateTime),

codes.CodeDescription

ORDER BY

Date_of_search,

searchstats.SSRequesterType,

codes.CodeDescription

;

```

Query: search_Requester_Type_Requester_Location_By_Date_Count @nasa_crest (MySQL)

Requester_Type	Location_Name	SatelliteID	Date_of_search	Counts
Public	Mexico	PreSat.org	2008-05-11	3
Public	Mexico	PharmaSat.org	2008-05-11	1
Student	Belgium	PharmaSat.org	2008-05-11	1
Student	Ecuador	PharmaSat.org	2008-05-11	1
Student	Falkland Islands (Malvinas)	PharmaSat.org	2008-05-11	1
Student	Saint Helena	PharmaSat.org	2008-05-11	3
Student	Saint Kitts and Nevis	PreSat.org	2008-05-12	2
Student	Saint Helena	PreSat.org	2008-05-23	8

SELECT searchstats.SSRequesterType AS Requester_Type, codes.CodeDescription AS Location_Name, searchstats.SSRequesterLocation AS SatelliteID, COUNT(*) AS Counts

Query time: 0.000s Record 1 of 8

To get the search requester Type & Location counts

SELECT

searchstats.SSRequesterType AS Requester_Type,

codes.CodeDescription AS Location_Name,

searchstats.SSRequesterLocation AS SatelliteID,

COUNT(*) AS Counts

FROM

searchstats ,

codes

WHERE

searchstats.SSRequesterLocation = codes.CodeID AND

codes.CodeType = 'Country'

GROUP BY

searchstats.SSRequesterType,

searchstats.SSRequesterLocation,

searchstats.SSRequesterLocation

;

[Query] Search_Requester_Type_Requester_Location_Count @nasa_crest (MySQL)

File Edit View Window

Run Stop Export Wizard New Load Save Save As Grid View Form View Memo

Query Builder Query Editor **Result** Explain

Requester_Type	Location_Name	SatelliteID	Counts
Public	Mexico	PharmaSat.org	1
Public	Mexico	PreSat.org	3
Student	Belgium	PharmaSat.org	1
Student	Ecuador	PharmaSat.org	1
Student	Falkland Islands (Malvinas)	PharmaSat.org	1
Student	Saint Helena	PharmaSat.org	3
Student	Saint Kitts and Nevis	PreSat.org	2
Student	Saint Helena	PreSat.org	8

SELECT searchstats.SSRequesterType AS Requester_Type, codes.CodeDe: Query time: 0.000s Record 1 of 8

To get the SessionID counts of the people who have logged into the website

SELECT

SsSessionID AS SessionID,

COUNT(*) AS Counts

FROM

searchstats

GROUP BY

SsSessionID

;

SessionID	Counts
1en0qf451ych0vn33sowszrr	1
1t1042yweksqmluwxaw1e355	1
i2tchf31eltgwkrwnqhugn22	1
jokkrrvzfgfmg45cwsnv1vj	1
m1qvkuao03g5nv45o32g4245	5
mukuwk45atcbw55kq4exv55	1
nrplwwq2yg55rh3mq4d5ocvq	1
nxah1w55cy1nf5fdwkfukj45	1
onbcmk455lfa2lixopatfxa1	4
tdhvlf55q40f5hf0pagdsl55	1
uwxebzzzj4or3255gyuilsyi	1
z3n5vzasn4ekj3yyr1tgou55	2

SELECT SsSessionID AS SessionID, COUNT(*) AS Counts F Query time: 0.000s Record 1 of 12

To get the Count of all the search that has been done on the database

SELECT

COUNT(*) AS Total_Number_Of_Search_Counts

FROM

searchstats;

Total_Number_Of_Search_Counts
20

SELECT COUNT(*) AS Total_Number_Of_Search_Cour Query time: 0.000s Record 1 of 1